

高级算法设计与分析：随机算法核心知识点考前冲刺手册

哈尔滨工业大学计算机科学与技术学院 (HIT CS&E) 内部复习参考

一、概率论基础与尾概率界 (Tail Bounds)

1. 期望的线性性质 (Linearity of Expectation) 必考重点

对于任意两个随机变量 X 和 Y （无论它们是否独立），以及任意常数 c ，均满足：

$$E[X + Y] = E[X] + E[Y]$$

$$E[cX] = \sum x \cdot \Pr[cX = x] = c \sum x' \cdot \Pr[X = x'] = c E[X]$$

考试提醒： 线性性质是求解复杂随机变量期望的终极利器，通常通过引入指示随机变量 (Indicator Random Variables) 将整体拆分为局部进行求解。

2. 典型实例：猴子打字机问题 高频考点

问题： 猴子在打字机上随机连续敲出长度为 10^9 的字母串，“proof” 平均出现多少次？

分析方法：

设 $X_i = 1$ 表示 “proof” 出现在位置 i 处，否则 $X_i = 0$ 。

“proof” 长度为 5，可能的位置共有 $10^9 - 5 + 1 \approx 10^9 - 4$ 个。

因为每个字母相互独立且均匀分布，故 $\Pr[X_i = 1] = 1/26^5$ ，从而 $E[X_i] = 1/26^5$ 。

由期望线性性质，总次数 $X = \sum X_i$ 的期望为：

$$E[X] = E[\sum X_i] = \sum E[X_i] = (10^9 - 4) \cdot (1/26^5) \approx 84 \text{ 次。}$$

3. 马尔可夫不等式 (Markov's Inequality) 必考重点

对于任意非负随机变量 X ，对任意 $t > 0$ ，恒有：

$$\Pr[X \geq t] \leq E[X] / t$$

推广形式： 对任意随机变量 X 和非负单调函数 $h: X \rightarrow \mathbb{R}^+$ ，有 $\Pr[h(X) \geq t] \leq E[h(X)] / t$ 。

特征： 马尔可夫不等式非常粗糙，它仅需要知道期望 $E[X]$ 即可给出上界。

4. 方差与常用分布方差公式 重要基础

离散随机变量 X 的方差定义为： $\text{Var}[X] = E[(X - E[X])^2] = E[X^2] - (E[X])^2$ 。

标准差定义为： $\delta[X] = (\text{Var}[X])^{1/2}$ 。

两点分布（伯努利分布）的方差：若 $\text{Pr}[X=1]=p$, $\text{Pr}[X=0]=1-p$ ，则：

$$E[X] = p, \text{Var}[X] = p(1-p)$$

5. 切比雪夫不等式 (Chebyshev's Inequality) 必考重点

对任意随机变量 X ，对任意 $t > 0$ 恒成立：

$$\text{Pr}[|X - E[X]| \geq t] \leq \text{Var}[X] / t^2$$

另一种表示（利用标准差）：令 $t' = t \cdot \delta[X]$ ，则 $\text{Pr}[|X - E[X]| \geq t \cdot \delta[X]] \leq 1 / t'^2$ 。

证明思路：令 $Y = (X - E[X])^2$ ，则 Y 是非负随机变量，且条件 $|X - E[X]| \geq t \Leftrightarrow Y \geq t^2$ 。对 Y 和 t^2 运用马尔可夫不等式即证。

6. 尾概率界 (Tail Bound) 分析方法对比 综合分析题点

在评估随机算法超过阈值的犯错概率时，通常有以下分析手段：

- **手段一（精确计数）**：计算直接繁琐，涉及极其复杂的组合数计算，实际中往往难以有效展开。
- **手段二（特征归结）**：计算其特征界限，形如 $\text{Pr}[X > t] < f(t, I)$ 。
 - **仅知期望**：采用马尔可夫不等式（通常界较松）。
 - **已知方差**：采用切比雪夫不等式（利用了二阶矩，界更紧）。
 - **已知更高阶矩/独立性**：采用切尔诺夫界（Chernoff Bound）。

二、数值随机算法 (Numerical Randomized Algorithms)

7. 蒙特卡罗投点法计算 π 值 基础实例

数学基础：向正方形随机投掷 n 个点，设 k 个点落入圆内。投掷点落入圆内的概率为 $(\pi r^2) / (4r^2) = \pi/4$ 。用 $k/n \approx \pi/4$ ，于是 $\pi \approx 4k/n$ 。

算法复杂度： $O(n)$ ，注意此处的 n 是随机样本量的大小，而非输入规模。

核心思考：精度随着随机样本大小 n 增加而增加。样本数 n 和精度之间可以通过强大数定律建立关联。

8. 随机算法近似计算定积分 重点理解

数学基础（大数定律）： 令 $f(x)$ 是区间 $[a, b]$ 上一组独立、同分布随机变量的任意密度函数。令 $g^*(x) = g(x)/f(x)$ ，则其期望满足 $E[g^*(\xi)] = \int g(x) dx = I$ 。由强大数定律，可用样本均值近似计算：

$$I' = (1/n) \sum g^*(\xi_i)$$

若令 $f(x) = 1/(b-a)$ ，则积分可以由如下形式近似计算： $I' = \frac{b-a}{n} \sum g(\xi_i)$ 。

三、随机选择与拉斯维加斯 (Las Vegas) 算法

9. 拉斯维加斯算法 (Las Vegas Algorithm) 的本质特征 必考重点

- **正确性：** 算法不会产生不正确的解。一旦算法得到问题的解，则该解百分之百是正确的。
- **随机性：** 算法运行过程的运行时间是不确定的（属于随机变量）。
- **运行机制：** 算法运行一遍可能无法找到解（返回失败）。通过反复运行算法，最终必然可以得到问题的正确解。

10. LazySelect 随机选择算法流程 高频考点

输入： $S = \{x_1, \dots, x_n\}$ ，整数 k 。

核心步骤：

1. 从 S 中独立、均匀、可放回地随机抽取 $n^{3/4}$ 个元素存入 R ，并在 $O(n)$ 内排序 R 。
2. 令支撑点排名 $x = (k/n)n^{3/4}$ 。
3. 设定安全界限： $l = \max(x - n^{1/2}, 0)$, $h = \min(x + n^{1/2}, n^{3/4})$ 。
4. 令边界元素 $L = \min(R, l)$, $H = \min(R, h)$ 。计算其在 S 中的排名 L_p, H_p 。
5. 过滤集合 $P = \{y \in S \mid L \leq y \leq H\}$ 。
6. **判断退出：** 若 $\min(S, k) \in P$ 且 $|P| \leq 4n^{3/4} + 1$ ，则排序 P 并返回正确解，否则重新跳转到步骤 1 运行。

11. LazySelect 算法单遍成功概率证明要点 必考大题型

定理： 算法执行一遍就可求出目标的概率是 $1 - O(n^{-1/4})$ ，即期望比较次数为 $O(n)$ 。

证明逻辑：

令随机变量 X 为 R 中小于等于 $\min(S, k)$ 的样本数。每一项样本满足伯努利分布，成功概率为 k/n 。

期望 $\mu_x = n^{3/4}(k/n) = x$ ，方差 $\sigma_x^2 \leq n^{3/4}/4$ ，标准差 $\sigma_x \leq n^{3/8}/2$ 。

如果条件(1)不成立（即边界无法包围目标），则意味着 $X < l$ 或 $X > h$ ，即 $|X - \mu_x| > n^{1/2}$ 。

由于 $2n^{1/8}\sigma_x \leq n^{1/2}$ ，应用切比雪夫不等式：

$$\Pr[|X - \mu_x| > n^{1/2}] \leq \Pr[|X - \mu_x| \geq 2n^{1/8}\sigma_x] \leq 1 / (2n^{1/8})^2 = O(n^{-1/4})$$

由于过滤失败的概率也是同样的数量级，单遍成功概率最终由互补事件公式确定为 $1 - O(n^{-1/4})$ 。

四、素数测试与蒙特卡罗 (Monte Carlo) 算法

12. 蒙特卡罗算法 (Monte Carlo Algorithm) 的本质特征 必考重点

- **运行时间：** 算法的运行时间是**固定的**（非随机变量）。
- **正确性：** 算法得到的解有可能发生错误。
- **单面错误蒙特卡罗 (MC1)：** 当问题答案为某特定解时（例如 Yes），算法总能给出 100% 正确的结果；但如果输出反向结果（例如 No），则该结论可能是错的。
- **双面错误蒙特卡罗 (MC2)：** 算法输出 Yes 或 No 时，都有可能犯错。

13. 素数测试随机算法设计 经典应用

核心机制： 随机选择 m 个数 $\{b_1, \dots, b_m\}$ 满足 $1 \leq b_i \leq N$ 。只要有一次测试使得测试函数 $W(b_i)$ 成立，算法直接返回“N是合数”；若全部不成立，则返回“N是素数”。

测试函数 $W(b)$ 的定义：

- (1) $b^{N-1} \not\equiv 1 \pmod N$ ，或
- (2) $\exists j$ 使得 $(N-1)/2^j = k$ 是整数，且 $1 < \gcd(b^k - 1, N) < N$ 。

概率结论： 回答“N是合数”时百分之百正确；回答“N是素数”时正确的概率为 $1 - 2^{-m}$ 。

14. 蒙特卡罗成功率放大技术 (Success Amplification) 必考重点

- **MC1 算法放大：** 重复运行 t 次。只要有一次判定发生了反例，就输出反例解。犯错概率（真解为 No 但输出 Yes）会以指数级衰减： $\Pr[\text{Error}] \leq (1 - \epsilon)^t \leq \delta$ 。可灵活建立重复遍数 t 与容错度 δ 的定量关系。
- **MC2 算法放大：** 重复运行 t 次，通过输出占多数的答案进行决策（多数表决制）。其错误概率同样随着迭代次数增大而快速向 0 收敛。

五、随机排序与舍伍德 (Sherwood) 算法

15. 舍伍德算法 (Sherwood Algorithm) 核心原理 必考重点

- **精髓：**对确定型算法进行随机化。其核心目的在于消除算法在最好实例和最坏实例之间的巨大差别。
- **特征：**舍伍德算法总能得到问题的正确解。
- **示例：**确定性快速排序在最坏实例下复杂度为 $O(n^2)$ 。通过Sherwood随机化（均匀等可能地选择划分轴元），成功将其期望时间复杂度平摊至 $O(n \log n)$ ，使其不再受极端恶劣输入数据分布的影响。

16. 随机快速排序期望复杂度的证明要点 必考大题型

核心证明思路：

设随机变量 $X_{ij} = 1$ 表示排序中第 i 小元素与第 j 小元素发生过比较，否则为 0。总比较次数期望为 $E[T(n)] = \sum \sum E[X_{ij}] = \sum \sum p_{ij}$ 。

要计算概率 p_{ij} ，考察元素集合 $S_{\{i \sim j\}} = \{S(i), \dots, S(j)\}$ （共 $j-i+1$ 个元素）。当且仅当算法在首次选中该集合中的任何元素作为划分轴元时，选中的恰好是边界 $S(i)$ 或 $S(j)$ ，这两者才会发生比较。若首先选到中间的元素，它们将被切分到两个独立的子树中，永不发生比较。因此：

$$p_{ij} = 2 / (j - i + 1)$$

利用调和级数求和公式展开：

$$E[T(n)] = \sum_{i=1}^n \sum_{j>i} \frac{2}{j-i+1} \leq 2n \sum_{k=1}^n (1/k) = O(n \log n)$$

六、最小割与高级成功率放大技术

17. Karger 随机边收缩算法 (Contraction) 重要考点

边收缩操作 $G/(x,y)$ ：用一个新节点代替节点 x 和 y ，原本连接到 x 或 y 上的边全部重定向到新顶点上，去掉自环。

CONTRACTION 算法流程：

1. $H = G$;
2. 当 $|H(V)| > 2$ 时执行：
3. 随机从 $H(E)$ 中等可能选择一条边 (x, y) ;
4. 进行边收缩操作 $H = H / (x, y)$;
5. 返回连接最后剩余两个元顶点的原始边集（即为一个割）。

特性：时间复杂度为 $O(n^2)$ 。单次运行能够成功找到图的最小割的下界概率非常低，为 $\Pr[\text{Success}] > 2/n^2$ 。

18. 边收缩算法单次成功率的严格证明 必考大题型

定理： 设图最小割大小为 k ，算法结束时，最小割中无一条边被收缩过的概率大于 $2/n^2$ 。

证明过程：

根据度数原理，若最小割大小为 k ，每个顶点的度数至少为 k ，则图的总边数至少为 $kn/2$ 。

第 1 步中选中最小割中边的概率至多为 $k/(kn/2) = 2/n$ 。即第 1 步安全的概率 $\Pr(A_1) \geq 1 - 2/n$ 。

第 i 步时，图还剩 $n-i+1$ 个节点，总边数至少为 $k(n-i+1)/2$ 。在前面都安全的前提下，当前步依然安全的条件概率为 $\Pr(A_i | \cap_{j=1}^{i-1} A_j) \geq 1 - 2/(n-i+1)$ 。

将这 $n-2$ 步的条件概率全部相乘：

$$\Pr(\text{Success}) \geq \prod_{i=1}^{n-2} (1 - \frac{2}{n-i+1}) = \frac{n-2}{n} \times \frac{n-3}{n-1} \times \dots \times \frac{2}{3} \\ = \frac{2}{n(n-1)} > \frac{2}{n^2}$$

致命弱点： 随着收缩深入，破坏概率（如最后一步为 $2/3$ ）剧烈攀升。

19. 概率放大策略：DetRan 算法与简单重复 高频考点

- **简单重复 Amplify1：** 重复独立运行 CONTRACTION 算法 $n^2/2$ 遍，取最小值。不成功概率降为 $(1 - 2/n^2)^{n^2/2} < e^{-1}$ 。但总时间复杂度飙升至 $O(n^4)$ ，丧失了实用价值。
- **DetRan 算法：** 当顶点数收缩到 $d(n) = n^{2/3}$ 时就立刻停止随机收缩，直接调用时间复杂度为 $O(d(n)^3)$ 的精确确定型最小割算法。此时 DetRan 算法的时间复杂度控制在 $O(n^2)$ ，而成功率大幅提升至 $\approx n^{-2/3}$ 。
- **Amplify2 组合拳：** 独立运行 DetRan 算法 $n^{2/3}$ 遍，总运行时间优化至 $O(n^{8/3})$ ，即可将成功率放大到稳定的大于等于 $1 - e^{-1}$ 。

20. 高级概率放大：RepTree 递归分支树算法 顶级重点

基本哲学： 顺应边收缩规律，实行“前期犯错率低，少重复；后期犯错率极高，多重复”的树状分叉重试策略。

RepTree(G) 算法流程：

1. 若顶点数 $n \leq 6$ ，直接调用确定型算法求解并返回。
2. 令收缩目标规模 $h = \text{ceil}(n / \sqrt{2})$ 。
3. 随机独立收缩 G 中 $n-h$ 条边得到缩减图 G1；
4. 随机独立收缩 G 中 $n-h$ 条边得到缩减图 G2；
5. 递归调用：S1 = RepTree(G1)；S2 = RepTree(G2)；
6. 返回二者的较小值：return min(S1, S2)；

考试核心结论：

- **时间递归方程：** $T(n) = 2T(n/\sqrt{2}) + O(n^2)$ 。根据主定理计算得总时间复杂度为 $O(n^2 \log n)$ 。
- **成功概率：** 单次运行获得正确解的概率高达 $\Omega(1/\log n)$ 。
- **最终放大：** 独立运行 RepTree 算法 $\log^2 n$ 遍，总时间开销为 $O(n^2 \log^3 n)$ ，即可将获得图的最小割的概率放大至几近于 1 发生（极度接近 1）。